

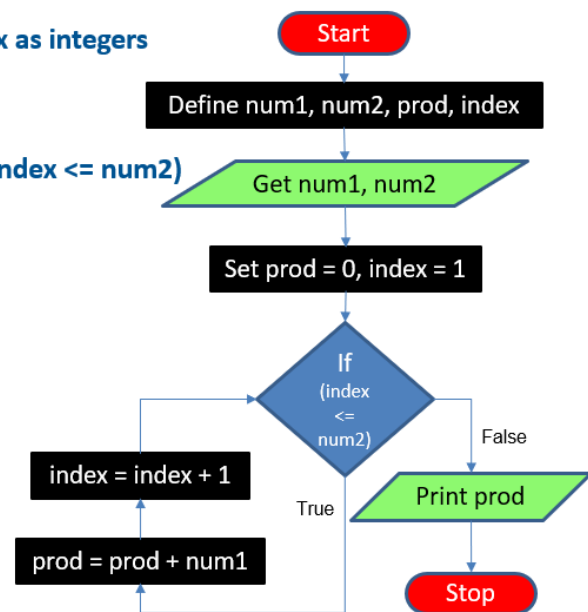
Activity – Day 4



Multiply 2 numbers without using *operator (9)

- Algorithm & Flowchart

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1. prod = prod + num1
 - 4.2. index = index + 1
5. Print prod
6. Stop



In this activity we are going to find faults in the given algorithm and flowchart above .

Mistakes in the algorithm line by line

1. Here, in line 1 every instruction is fine and true, cause we have to first declare some variables like sum, index, num1, num2 as integers to perform mathematical operations upon them.
2. Second line is also quite correct as we have to consider num1 and num2 here , as the problem is itself about multiplying two numbers.
3. Same goes for third line.
4. In fourth line a mistake has occurred which is the absence of while loop instead of until cause using until we have changed the logic completely .
5. For negative numbers , there exists no logic about what to do. There is only logic if user enters positive num1 and num2.
6. There is also no algorithm for if someone enters 0 , as the program will still tries to loop.
7. There is also no algorithm to how to minimize the number of additions if we want product of two numbers.

Correct Algorithm

1. Define num1, num2, prod and index as integers.
2. Get num1 and num2
3. If num1 == 0 or num2 == 0 then

 Prod = 0

 Print Prod

 Stop

4. Set Prod = 0
5. Set Sign = 1
6. If num1 < 0 then

 Num1 = |Num1|

 Sign = -Sign

7. If num2 < 0 then

 num2 = |num2|

 Sign = -Sign

8. If num1 < num2 then

 Smaller = num1

 Larger = num2

Else then

 Smaller = num2

 Larger = num1

9. For index = 1 to smaller do

 Prod = prod + larger

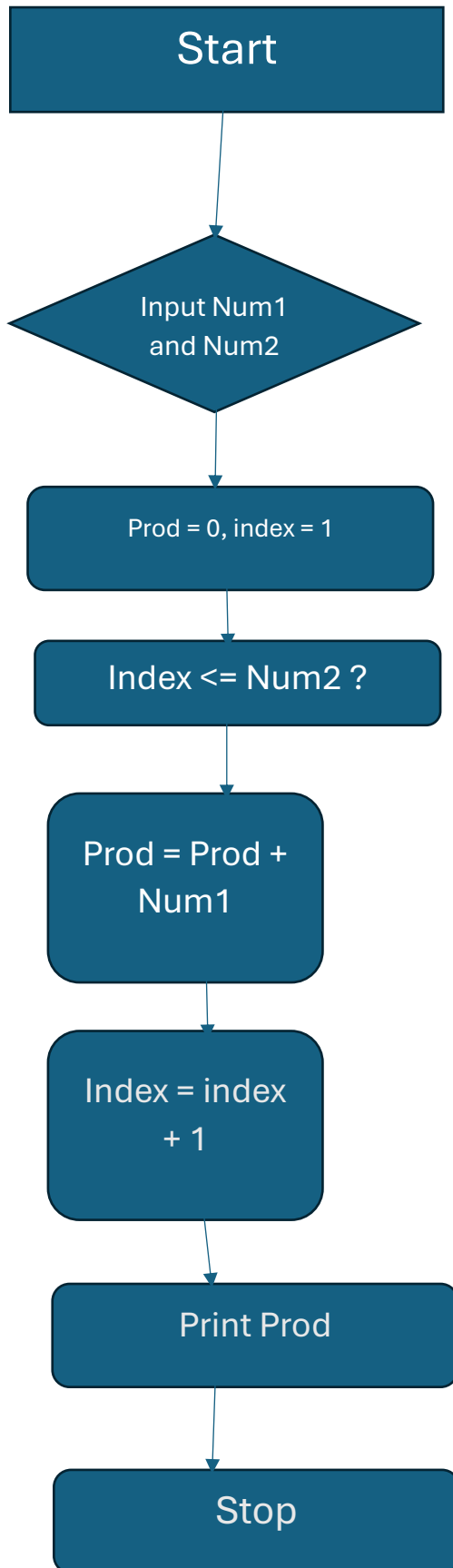
10. Prod = Prod * sign

11. Print prod

12. stop

Mistakes in the flowchart

1. Here in the decision diamond block occurs a contradiction , As in the algorithm we have written repeat until index $\leq$ num1, but here it had been stated directly if index $\leq$ num1.
2. The flowchart doesn't have any info if there input comes as negative num1 or num2 thus our loop falls for negative values.
3. Same happens when some input either num1 or num2 comes as 0, the loop falls.
4. The flowchart doesn't compare which is the minimum value (num1 or num2) instead randomly assign operation on num1. Because to find product of two numbers in minimum attempts of addition , we should compare which number or input if say is smaller .



Our program

```
#include <stdio.h>
int multiply(int a, int b){
    int product = 0;
    int positive = 1;

    if (a < 0) {
        a = -a;
        positive = -positive;
    }
    if (b < 0) {
        b = -b;
        positive = -positive;
    }

    int smaller = (a < b) ? a : b;
    int larger = (a < b) ? b : a;

    for (int i=0; i<smaller; i++){
        product += larger;
    }
    if (positive < 0)
        product = -product;
    return product;
}

int main(){
    int x, y;
    printf("Enter the two numbers : ");
    scanf("%d %d", &x, &y);

    int result = multiply(x, y);
    printf("Product: %d\n", result);

    return 0;
}
```

Also , we had verified it using the test cases given at that time.